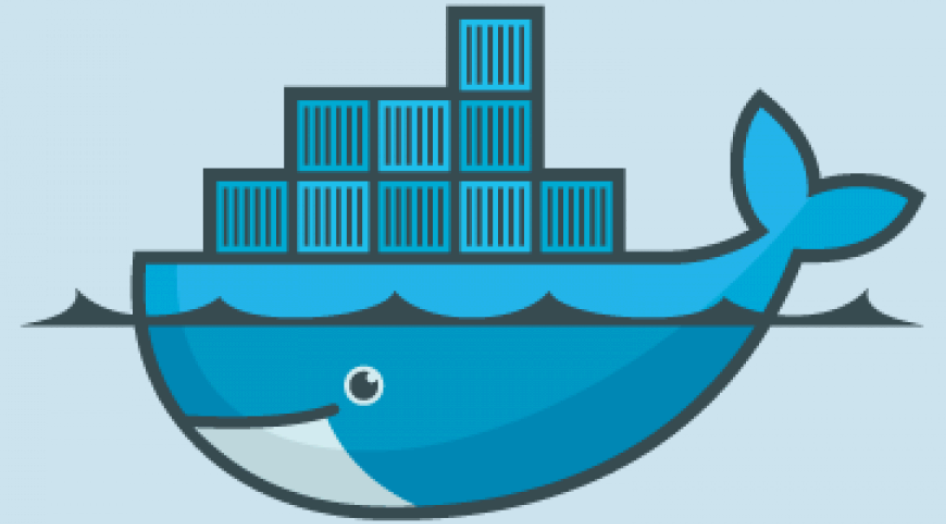


Intro to



docker

Agenda

- **Recap of last session**
- **Dockerfile**
- **Building images**
- **Docker environment**
- **Entrypoint**
- **Docker Networking**
- **Introduction to YAML**
- **Docker Compose**

Recap of last session

- We learned about Docker and how it is used to create containers.
- We learned Docker commands and how to run containers.

Docker Commands

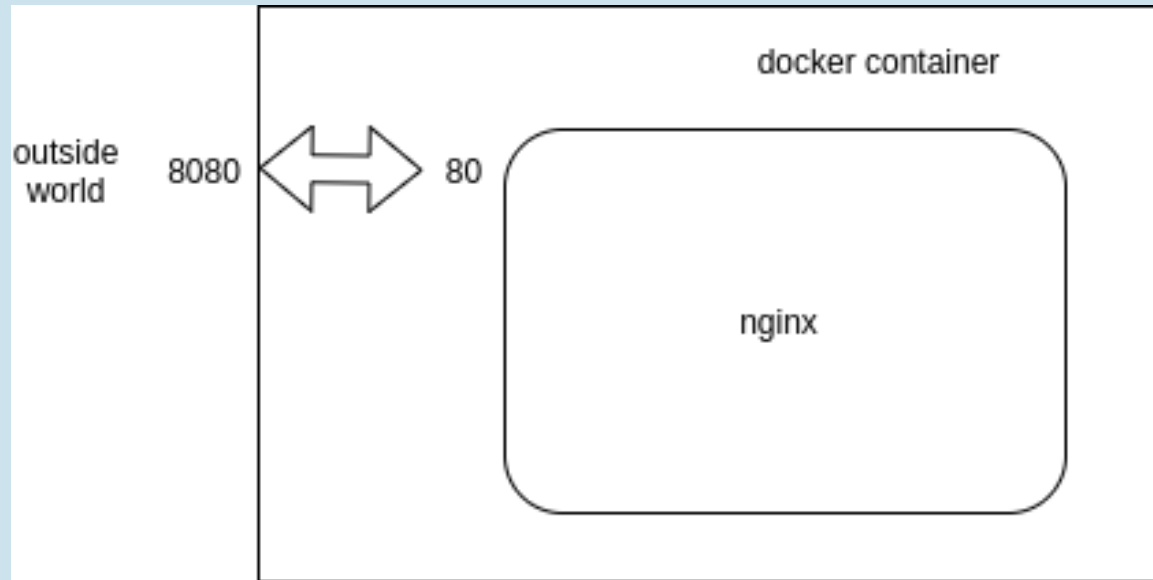
- `docker run <image_name>` : Runs a container from an image.
 - `docker run --name <container_name> <image_name>` : Runs a container with a custom name.
 - `docker run -d <image_name>` : Runs a container in detached mode.
- `docker ps` : Lists all running containers.
 - `docker ps -a` : Lists all containers, running or stopped.
- `docker stop <container_name>` : Stops a running container.
- `docker start <container_name>` : Starts a stopped container.
- `docker rm <container_name>` : Removes a stopped container.
- `docker rmi <image_name>` : Removes an image.

Docker Commands (cont.)

- `docker exec -it <container_name> <command>` : Executes a command in a running container.
- `docker pull <image_name>` : Pulls an image from Docker Hub.
- `docker images` : Lists all images on the host.

Port Binding

- If we have a web app or server running in a container it will be given an IP on the closed Docker network, so anything outside the container can't access it, to solve this we need to use port binding (mapping) which is binding or mapping the container's port to the host's port to make the service available outside the container



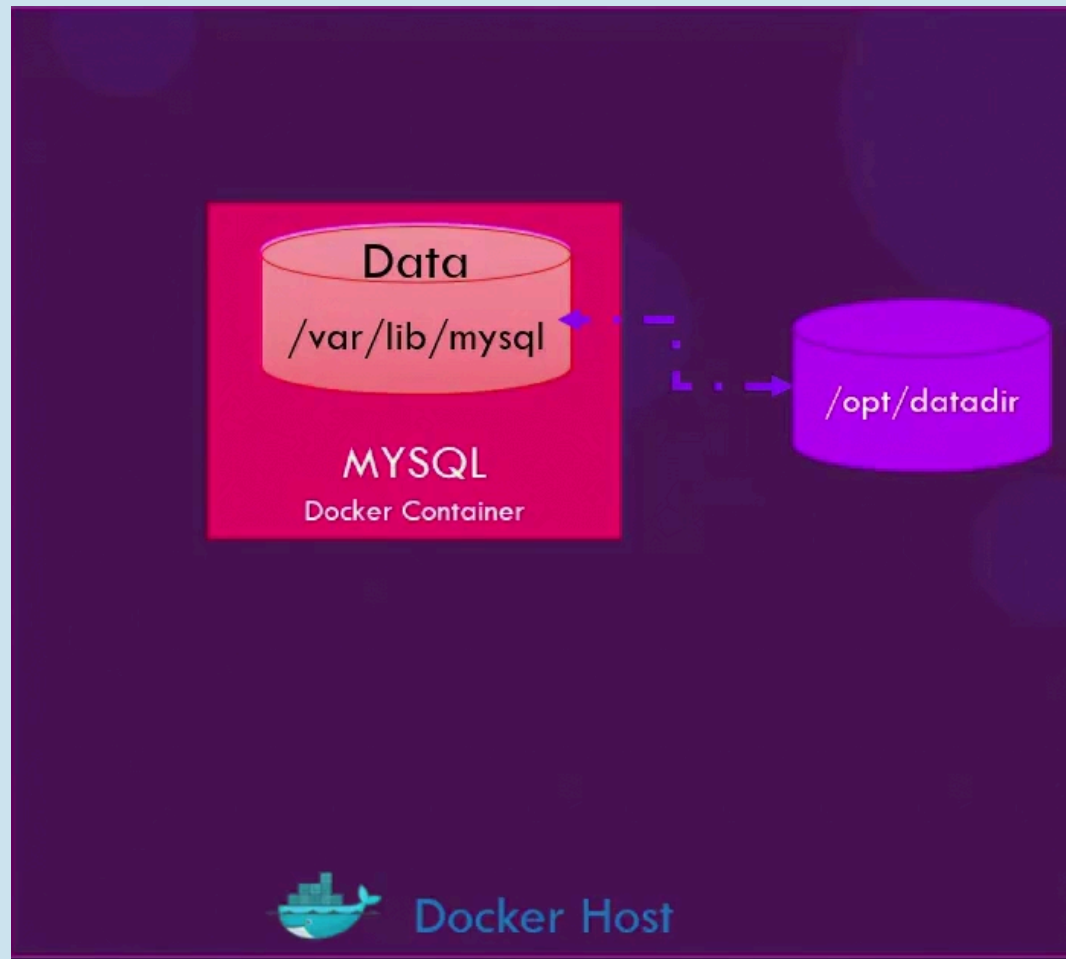
- nginx is a web server which runs on port 80, if we try to access the localhost at port 80 we will get nothing because port 80 has nothing on it now, we bind port 80 in the container to port 8080 on the host so now accessing the localhost on port 8080 we can access our server.

- Port mapping is done using the `-p` option with `docker run`
- `docker run -p <host_port>:<container_port> <image_name>`
- Note that only one service can run on this host port.

Volume Mapping

- Containers have their filesystem isolated from the host, so data stored and modified in the container aren't saved on the host.
- Let's say we have a database container, all data stored in this DB is stored in the container, if this container is removed, all data on the database is lost.
- We can port a volume by using `-v` option with `docker run`
- Example: `docker run -v tmpDirectory/:/tmp/db/ <image_name>`

- To solve this issue we use volume mapping, where as in port mapping, we map a volume on our container to a volume on our host so data are saved on host and independent of the container in a way.



Configure resource constraints

- By default, a container has no resource constraints and can use as much of a given resource as the host's kernel scheduler allows
- Docker provides ways to control how much memory, or CPU a container can use, setting runtime configuration flags of the `docker run` command
- It achieves this by using the `cgroups` feature of the Linux kernel

Configure resource constraints

- Ex. Limiting the cpu usage of a container to 50%

```
    
❯ docker run -it --cpus=.5 ubuntu bash
```

- Ex. Limiting the memory usage of a container to 1GB

```
    
❯ docker run -it --memory=1G ubuntu nginx
```

Images

- As we previously said, an image is a packaged application containing the source code and all dependencies of an application.
- We will look now at how to make our own images.

But, why would we want to create our own images?

- There are many reasons we might need that including:
 - Sometimes we can't find the service we want as an image
 - We want our application to be dockerized for ease of shipping and deployment

OK, so how do we go about creating our own images?

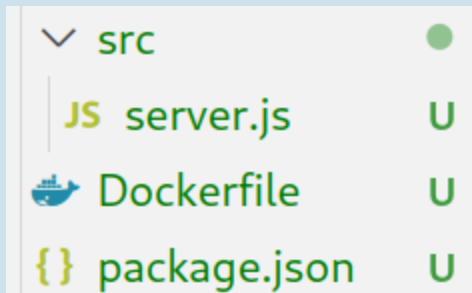
- To take our created application and package it into a Docker image we need to create a "definition" on how to build an image from our app, in other words we need to write instructions on how to build our app.
 - This "definition" is written in a text file called `Dockerfile`
 - Dockerfile contains commands to assemble an image
 - Docker can build images by reading those commands

We will use a simple Node.js app, we don't need to look into the specifics of how it is written, but we will focus on its output and how Docker affects it.

The application basically listens on port 3000 and prints welcome when accessed from the browser, also we have a json file which has dependencies for the library we are using.

Dockerfile

To start building our image we first need to create our Dockerfile, most text editors recognize this file



Structure of Dockerfile

- Dockerfile start from a parent image or "base image".
- It's a Docker image that your image is based on.
- We choose the base image depending on the tools we need to have.
 - For example here we need a base image that has `npm` installed, `npm` is a package manager for JavaScript and it installs most Node.js libraries.
- Dockerfile consists of instructions and arguments, we will look at some of these instructions as we go on building our server.

FROM

First instruction we have is `FROM`, all Dockerfiles *must* start with a `FROM` instruction.

`FROM` specifies the base image we are building our image on.

Base images are Linux images most of the time, in our case here we will also need a Linux image to base on but also we need to have `npm` installed on it so we can install any libraries needed for the server.

One such image is `node:19-alpine`, this image is a Node.js image based on the `alpine` image and has `npm` installed, this will do fine for us.

```
1 Dockerfile +  
1 FROM node:19-alpine  
~  
~  
~  
~  
~  
~
```

RUN

If we want to install a library using `npm` we use `npm install <package_name>` and on our image we will need to install the library we need for our app which is `express`.

To execute any command in a shell *inside* the container environment

So here we use `RUN npm install` to have npm install all dependencies for our app.

```
1 Dockerfile +  
1 FROM node:19-alpine  
2  
3 RUN npm install
```

COPY

OK so we now have our base image and installed our dependencies, are we ready to run the script now? Well, no.

We still don't have our app's files in the container, they are on our host system only.

To put our app in the container we use the `COPY` instruction.

- We can copy individual files like `package.json`

```
1 Dockerfile +  
1 FROM node:19-alpine  
2  
3 COPY package.json /app/  
4  
5 RUN npm install
```

- We can also copy entire folders.

```
1 Dockerfile +  
1 FROM node:19-alpine  
2  
3 COPY package.json /app/  
4 COPY src /app/  
5  
6 RUN npm install
```

Important thing to notice: The `/` after `app` is important as it tells Docker to create this folder if doesn't exist in the container.

`RUN` is executed in the container while `COPY` is executed on the host.

WORKDIR

Now are app's files is in the container but are we?

By default we are in the root directory of the container `/` while we copied our app to `/app`, we need to go to `/app`, if we were on the host we could use `cd`, well in Dockerfile we use `WORKDIR` and give it the directory we want as our working directory for all following commands.

```
1 Dockerfile +  
1 FROM node:19-alpine  
2  
3 COPY package.json /app/  
4 COPY src /app/  
5  
6 WORKDIR /app  
7  
8 RUN npm install
```

CMD

So we have chosen a base image, copied app files, set our working directories and installed dependencies, we are now ready to finally ready to run our app.

To do that, we use **CMD**

`CMD` has a specific syntax which goes as the following:

```
CMD ["command", "parameter"]
```

To run a server with Node.js we use `node server.js` command, so we pass it in the Dockerfile with `CMD`

```
1 Dockerfile +
1 FROM node:19-alpine
2
3 COPY package.json /app/
4 COPY src /app/
5
6 WORKDIR /app
7
8 RUN npm install
9
10 CMD ["node", "server.js"]
```

Note that `CMD` is the last instruction to run in the Dockerfile and it executes when the container starts.

Also note that we can have *one* `CMD` instruction in the Dockerfile.

CMD vs RUN

OK so we may notice that both `RUN` and `CMD` are used to execute terminal commands, so what is the difference between them?

`RUN` is an image build step, after each `RUN` instruction is committed to the container image, we can have many `RUN` instructions as we want.

`CMD` executes by default when a container from the image is started, there can be only one `CMD` in the Dockerfile.

Building the image

Now that we have written our Dockerfile, we have given Docker instructions to make an image.

To build an image we use the `build` command.

We use the `-t` option to give our image a name and optionally a tag.

```
[osc@navi Server Example]$ docker build -t test-server .  
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.  
            Install the buildx component to build images with BuildKit:  
            https://docs.docker.com/go/buildx/
```

```
Sending build context to Docker daemon 6.656kB
```

```
Step 1/6 : FROM node:19-alpine
```

```
---> e2a8cc97f817
```

```
Step 2/6 : COPY package.json /app/
```

```
---> 10ea6b2d300a
```

```
Step 3/6 : COPY src /app/
```

```
---> 5aa45b9cb66e
```

```
Step 4/6 : WORKDIR /app
```

```
---> Running in 0cf50c5387e7
```

```
Removing intermediate container 0cf50c5387e7
```

```
---> 1749fef993d6
```

```
Step 5/6 : RUN npm install
```

```
---> Running in 149773639f73
```

added 62 packages, and audited 63 packages in 5s

11 packages are looking for funding

run `npm fund` for details

found 0 vulnerabilities

npm notice

npm notice New major version of npm available! 9.6.3 -> 10.4.0

npm notice Changelog: <<https://github.com/npm/cli/releases/tag/v10.4.0>>

npm notice Run `npm install -g npm@10.4.0` to update!

npm notice

Removing intermediate container 149773639f73

---> 521833200932

Step 6/6 : CMD ["node", "server.js"]

---> Running in 28a403b7af9f

Removing intermediate container 28a403b7af9f

---> c1db14580399

Successfully built c1db14580399

Successfully tagged test-server:latest

Layered architecture

When Docker builds images, it does so in a layered architecture.

Each instruction creates a new layer within the Docker image with just the changes from the previous layer.

So the `FROM` instruction is our first layer, the first `COPY` is another layer which builds on the first and so on.

Entrypoint

As we previously said, containers are meant to run a specific task and to exit after completing it.

Linux containers by default don't have a process at the start so they exit immediately. This can be solved by appending commands to the `run` command.

Say what if we want an image to run a certain command always when it starts? We can build it and add a `CMD` instruction with the command we want.

For example, we want a alpine container that always runs `sleep 5` when it starts, so we make an image named `alpine-sleeper`

```
1 Dockerfile
2
3 FROM alpine
4
5 CMD ["sleep", "5"]
```

After building it and making a container from it, it will run `sleep 5` by default.

Now this image is called `alpine-sleeper` so it makes sense that it will always run `sleep`, so appending `sleep` to it in `run` isn't needed really.

Can we make it so that we pass the number of seconds we want it to sleep directly?

- Ex: `docker run alpine-sleeper 10` runs `sleep 10`.

This is where the instruction `ENTRYPOINT` comes in. `ENTRYPOINT` is like `CMD` as you can specify the program that will run when the container starts and whatever we append to the `run` command is passed as an argument.

```
1 Dockerfile +  
1 FROM alpine  
2  
3 ENTRYPOINT ["sleep"]
```

We should also add a `CMD` instruction after `ENTRYPOINT` as a failsafe, as if we don't append an argument to the `run` command an error will arise.

```
1 Dockerfile +
  1 FROM alpine
  2
  3 ENTRYPOINT ["sleep"]
  4
  5 CMD ["5"]
```

Lastly, we can specify a custom entrypoint by adding the `--entrypoint` option to `run`

```
osc@navi ~]$ docker run --entrypoint cat alpine-sleeper /etc/hostname
b747043ec769
osc@navi ~]$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
b747043ec769	alpine-sleeper	"cat /etc/hostname"	2 seconds ago	Exited (0) 1 second ago		happy_curie

Docker inspect

Every image is built using a Dockerfile, we can view how it is constructed in detail by inspecting the image using `docker inspect <image>`

The output of `inspect` isn't the exact Dockerfile but some of the instructions can be found in the output, for example, the `CMD` argument, size of image, working directory and other useful data.

From the things we can see in the `inspect` output is something called `Env`

Docker Environment

A Docker container is as we said an isolated environment, so it has its own variables that define important paths, variables, configurations in it.

We can pass environment variables to a container using the `-e` or `--env` options

```
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS          NAMES
[osc@navi ~]$ docker run -d --name test -p 27017:27017 -e MONGO_INITDB_ROOT_USERNAME=root -e MONGO_INITDB_ROOT_PASSWORD=password mongo
64befbeded11afa31efbd5567654764cccd4d88e8f30480e7b036f25c63bc21f
```

Hands-on

1. `alpine-bash`

- Use `alpine` as the base image.
- Install `bash` using the following command: `apk add bash`

2. `alpine-server`

- Use `alpine-bash` as your base image.
- Download and copy the `server.sh` file and copy it to `/opt/myServer` directory in your image.
- Install `socat` package.
- Go to the directory containing the script.
- Have the script start when we make a container of this image.
- Run a container from the image and go to `localhost:1984` on your host

Break



Docker Networking

- When you install docker it creates 3 networks by default:
 - bridge
 - none
 - host

```
> docker network ls
NETWORK ID        NAME        DRIVER        SCOPE
8aaaab951d63     bridge     bridge        local
771ba4d28add     host       host          local
679ac1f90551     none      null          local
```

- to attach a container to a specific network you can use the `--network` option

```
docker run --network <network-name> <image-name>
```

Docker Networking - default bridge network

- private internal network created by docker on the host
- all containers attached to this network by default and they get an internal IP address
- containers can communicate with each other using their internal IP addresses
- to access any container inside this network from outside the network you can use the port mapping

```
docker run -p <host-port>:<container-port> <image-name>
```

Docker Networking - default bridge network

```
> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
7368406311ba	alpine	"sh"	5 minutes ago	Up 5 minutes		ali
cfa4123e56fb	alpine	"sh"	6 minutes ago	Up 5 minutes		karim

```
> bridge link
```

```
10: vethea4c10d@if9: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 master docker0 state forwarding priority 32 cost 2  
12: vethe91da1c@if11: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 master docker0 state forwarding priority 32 cost 2
```

Docker Networking - default bridge network

```
"ConfigOnly": false,
"Containers": {
  "7368406311ba2a55d57a526b5b20396534366568aaecee7cdebe2a19fdd171d1": {
    "Name": "ali",
    "EndpointID": "d0ca380c7f6876ce520ac14021e5a034cfdb3183394c864157017f924c8720ca",
    "MacAddress": "02:42:ac:11:00:03",
    "IPv4Address": "172.17.0.3/16",
    "IPv6Address": ""
  },
  "cfa4123e56fb29527a423e66c7fd8af9b840bee148c9a6eeb4868d1e04062c85": {
    "Name": "karim",
    "EndpointID": "f913671c488c9e9129a1a646367946473a14c0fd90ef3fa8cf69d3bd6fac9c19",
    "MacAddress": "02:42:ac:11:00:02",
    "IPv4Address": "172.17.0.2/16",
    "IPv6Address": ""
  }
},
}
```

Docker Networking - none network

- When you run a container with the `--network none` option, the container does not get any network access
- This means that the container cannot communicate with the host or any other containers
- This is useful when you want to run a container in complete isolation
- You can still access the container using the `docker exec` command

Docker Networking - none network

```
> docker run -it --network=none alpine sh
/ # ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
/ #
```

Docker Networking - host network

- It is one of the 3 default networks created by docker
- When you run a container with the `--network host` option, the container shares the host's network stack
- This means that the container does not get its own network namespace, and it uses the host's network stack
- This removes the network isolation between the container and the host

Docker Networking - host network

```
> docker run ahmedyasser9/osc-server  
2024/02/04 10:23:07 Listening on port 7000...
```

```
> curl localhost:7000/osc  
curl: (7) Failed to connect to localhost port 7000 after 0 ms: Couldn't connect to server
```



Docker Networking - host network

```
> docker run --network=host ahmedyasser9/osc-server  
2024/02/04 10:24:20 Listening on port 7000...
```

```
> docker run --network=host ahmedyasser9/osc-server  
2024/02/04 10:24:23 Listening on port 7000...  
2024/02/04 10:24:23 [server.ListenAndServe, listen tcp :7000: bind: address already in use]
```



Docker Networking - user-defined bridge network

- You can create your own bridge network using the `docker network create` command

```
docker network create <network-name>
```

- Containers attached to the same network can communicate with each other using their container names or their internal IP addresses

Docker Networking - user-defined bridge network

```
> docker network create session3
cd5c92ad9f56948b931910c69a88065440bba2efcf9db66d11c20f2bf6484af7
> docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
8aaaab951d63        bridge              bridge              local
771ba4d28add        host                host                local
679ac1f90551        none                null                local
cd5c92ad9f56        session3            bridge              local
```

Docker Networking - user-defined bridge network

```
"Containers": {  
  "83eaf60b5d28f7e2d6c256c23f4fafed72e9226baa38a830eb097580f1954c0c": {  
    "Name": "ehab",  
    "EndpointID": "e19e3117aad060eda9e87e481a2bc40138b7705a2b599878fddc413f9fde8fa1",  
    "MacAddress": "02:42:ac:14:00:03",  
    "IPv4Address": "172.20.0.3/16",  
    "IPv6Address": ""  
  },  
  "8dc3de810d4e2900b5dc59840c8768be61bbdce445b2d5b8f5a1eb20c30d88bb": {  
    "Name": "nour",  
    "EndpointID": "c0511aff3c28f7387abeab705bc772170a77c1974e97ad7f0d9ec77b71bfbc1e",  
    "MacAddress": "02:42:ac:14:00:02",  
    "IPv4Address": "172.20.0.2/16",  
    "IPv6Address": ""  
  }  
},
```

Docker Networking - user-defined bridge network

```
> docker attach nour
/ # ping ehab
PING ehab (172.20.0.3): 56 data bytes
64 bytes from 172.20.0.3: seq=0 ttl=64 time=0.205 ms
64 bytes from 172.20.0.3: seq=1 ttl=64 time=0.139 ms
64 bytes from 172.20.0.3: seq=2 ttl=64 time=0.145 ms
^C
--- ehab ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.139/0.163/0.205 ms
/ #
/ #
/ # ping 172.20.0.3
PING 172.20.0.3 (172.20.0.3): 56 data bytes
64 bytes from 172.20.0.3: seq=0 ttl=64 time=0.189 ms
64 bytes from 172.20.0.3: seq=1 ttl=64 time=0.134 ms
64 bytes from 172.20.0.3: seq=2 ttl=64 time=0.133 ms
^C
--- 172.20.0.3 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.133/0.152/0.189 ms
/ #
```

Introduction to *yaml*

Introduction to yaml

- YAML stands for "YAML Ain't Markup Language" or "Yet Another Markup Language"
- YAML is a human-readable data serialization standard that can be used in conjunction with all programming languages and is often used to write configuration files.
- YAML is a superset of JSON, which means that any valid JSON file is also a valid YAML file.

Introduction to yaml - key-value pairs

- Keys are separated from their values by a colon followed by a space.
- Key-value pairs are separated by a new line.
- Example

```
name: John  
age: 30
```

Introduction to yaml - Data Types

- Scalars
 - Strings
 - Numbers
 - Booleans
 - Nulls
- Collections
 - Arrays
 - Dictionaries

Introduction to yaml - Scalars

- Strings may be plain or quoted

```
name: John
```

- Numbers

```
age: 30  
grade: 9.5
```

- Booleans

```
isStudent: true
```

- Nulls

```
address: null
```

Introduction to yaml - Collections

- Arrays

```
fruits:  
  - Apple  
  - Banana  
  - Orange
```

```
fruits: [Apple, Banana, Orange]
```

- Dictionaries

```
person:  
  name: John  
  age: 30
```

```
person: { name: John, age: 30 }
```

Introduction to yaml - Syntax

- Indentation
 - YAML uses indentation to indicate nesting of elements.
 - The number of spaces is not fixed, but it should be consistent throughout the file.

Introduction to yaml - Syntax

- Example 1

```
person:  
  name: John  
  age: 30  
  address:  
    city: New York  
    state: NY
```

Introduction to yaml - Syntax

- Example 2

```
person:  
  - name: John  
    age: 30  
    address:  
      city: New York  
      state: NY  
  - name: Jane  
    age: 25  
    address:  
      city: Los Angeles  
      state: CA
```

Introduction to yaml - Notes

- Dictionary is an unordered collection where as lists are ordered collection.
- This means that in the following example

```
person:  
  name: John  
  age: 30
```

is the same as

```
person:  
  age: 30  
  name: John
```

Introduction to yaml - Notes

- While in the list the order matter

```
fruits:  
  - Apple  
  - Banana  
  - Orange
```

is not the same as

```
fruits:  
  - Banana  
  - Apple  
  - Orange
```

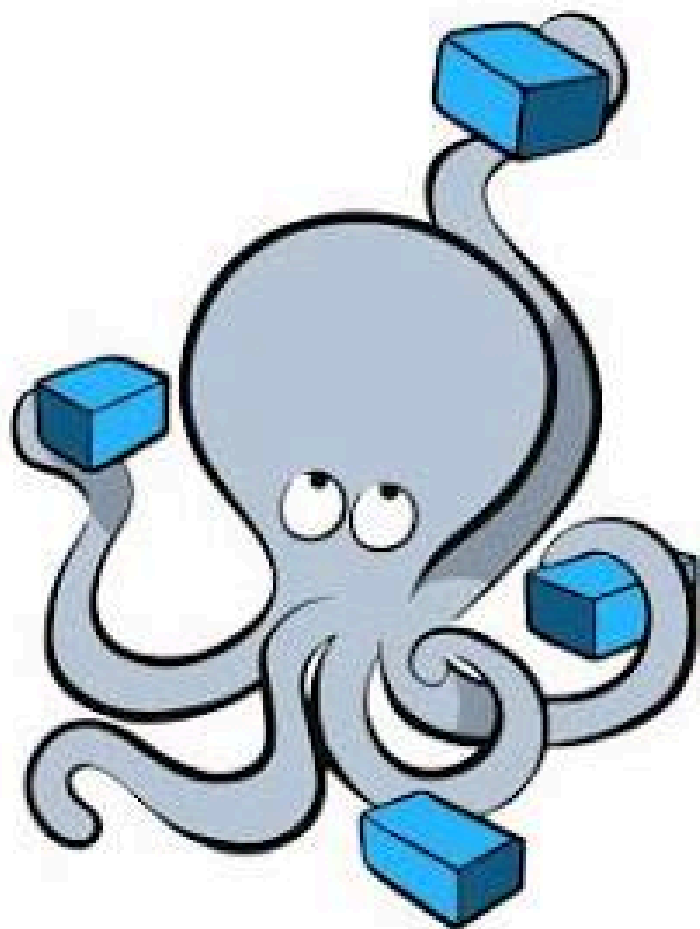
Introduction to yaml - Notes

- Any line beginning with a `#` is a comment and will be ignored by the parser.

```
# This is a comment
person:
  name: John
  age: 30
```

Break





docker
Compose

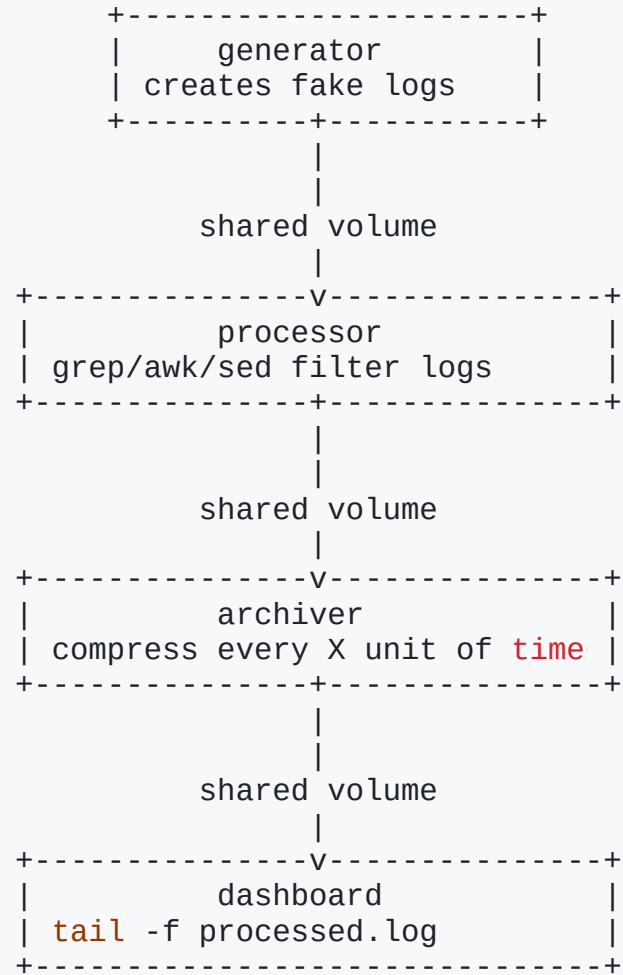
Docker Compose

- Docker Compose is a tool for defining and running multi-container Docker applications.
- With Compose, you use a YAML file to configure your application's services.
- Then, with a single command, you create and start all the services from your configuration.

Docker Compose - Example

- Suppose we have a log processing pipeline that consists of three services:
 - A log generator that generates logs and writes them to a file.
 - A log processor that reads the logs from the file, filters them, and writes the filtered logs to another file.
 - An archiver that archives and compresses the filtered logs.
 - A dashboard that tracks the filtered logs and displays them in the terminal.

Docker Compose Network



- We have our scripts for each service and their Dockerfiles, we will first start our app using basic Docker commands.

1. Build the images

```
docker build -t log-generator generator/
```

```
docker build -t log-processor processor/
```

```
docker build -t log-archiver archiver/
```

```
docker build -t log-dashboard dashboard/
```

2. Run the containers

```
docker run \  
  --name generator \  
  -e LOG_INTERVAL=1 \  
  -v $(pwd)/shared:/shared \  
  log-generator
```

```
docker run \  
  --name processor \  
  -e FILTER_LEVELS=ERROR \  
  -v $(pwd)/shared:/shared \  
  log-processor
```

```
docker run \  
  --name dashboard \  
  -v $(pwd)/shared:/shared \  
  log-dashboard
```

```
docker run \  
  --name archiver \  
  -e ARCHIVE_INTERVAL=20 \  
  -v $(pwd)/shared:/shared \  
  log-archiver
```

Ok, so what if we have 6 services? 10? or maybe 20?

- As we have seen with just 4 services this was a hassle, and if we needed to change an environment variable or a volume mapping we would have to stop our container, remove it and run a new one with the new configs.
- This is where docker compose comes in. let's try again using compose.

- A docker compose file is a YAML file that carries configurations for our multi container app.

- In the `service` key, we specify our container options, like what image to use, the name of the container, environment variables, ports and volumes mapped and many other configs.

Docker Compose - Versions

- Version 1
 - The original version of Compose
 - It is not recommended for production use
 - If you want to deploy containers on different network other than default bridge network, there is no way to specify that in version 1
 - If you have a dependency or start of order between services, there is no way to specify that in version 1

Docker Compose - Versions

- Version 2
 - From version 2 and up you must specify the version of the Compose file format being used
 - It is the most widely used version
 - It supports `networks` and `depends_on` options
 - It is the default version
 - All containers are listed under `services` key
 - Docker automatically create a dedicated bridge network for the services, and then attaches all containers to that new network, so all containers can communicate to each others using the service name, so you don't need to use `--link` option

Any questions?

Resources

- [Docker Docs](#)
- [Helisnki Docker Course](#)
- [Docker tutorial](#)
- [Docker Compose Docs](#)
- [Docker Compose tutorial](#)

Thank You!